Software Architecture and Design Specification (SAD)

Digital Library and Booking System (DLBS)

Project: Digital Library and Booking System (DLBS)

Version: 1.0

Authors: Mohammed Sinan M T, Mallela Rishyendra, Nagalapuram Sai Mukesh Reddy, Marreddy Rushieswar Reddy

Date: 30-10-2026

Status: Submitted (baseline before Sprint 1)

Revision history

Version	Date	Author	Change summary
1.0	30-10-2026	Team DLBS (all four members)	Baseline architecture and design for Sprint 1: components, data model, sequence diagrams, API design, security architecture

Approvals

Role	Name	Signature / Date
Architecture owner	Mohammed Sinan M T	
Scrum Master	Mallela Rishyendra	
QA Lead	Marreddy Rushieswar Reddy	
Course Coordinator		

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Audience 3
1.4 Definitions 3
2. Document overview 3
2.1 How to use this document 3
2.2 Related documents 3
3. Architecture 4
3.1 Goals and constraints 4
3.2 Stakeholders and concerns 4
3.3 Component (UML) diagram 4
3.4 Component descriptions 8
3.5 Chosen architecture pattern and rationale 8
3.6 Technology stack and data stores 8
3.7 Risks and mitigations 9
3.8 Traceability to requirements 9
3.9 Security architecture 10
4. Design 10
4.1 Design overview 10
4.2 UML sequence diagrams 11
4.3 API design 15
4.4 Error handling, logging and monitoring 16
4.5 UX design 16
4.6 Open issues and next steps 16
5. Appendices 17
5.1 Glossary 17
5.2 References 17
5.3 Tools 17

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Digital Library and Booking System. It explains how the system is divided into components, why that structure was chosen, how the components talk to each other, and how the main flows work. Developers build from it, and the QA Lead uses it to plan integration tests.

1.2 Scope

It covers the four application modules (Book Catalog, Users and Reading Tracking, Booking and Borrowing, Recommendation Engine), the web UI, the REST API layer, the shared database, and the CI/CD pipeline that builds, tests and deploys the application.

1.3 Audience

The development team, the QA Lead, the Scrum Master, future maintainers, and the course instructor and teaching assistants.

1.4 Definitions

Term	Meaning
ADR	Architecture decision record: one decision, its options and the reason for the choice
Modular monolith	One deployable application split into modules with clear interfaces
ORM	Object-relational mapper (SQLAlchemy) that turns Python classes into tables and queries
JWT	JSON Web Token, the signed login token
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TF-IDF	Term frequency – inverse document frequency text vectors
SD-n	Sequence diagram number n in section 4.2
TLS / HTTPS	Transport Layer Security; the deployed host serves the app only over HTTPS

2. Document overview

2.1 How to use this document

Section 3 gives the architecture: goals, stakeholders, the component and class diagrams, the chosen pattern, the technology stack, risks, traceability to requirements and the security architecture. Section 4 gives the design: module internals, sequence diagrams for three key flows, the API design, error handling and monitoring, and the UX design.

2.2 Related documents

- Software Requirements Specification (SRS) v1.0, including the RTM in section 8.
- Software Test Plan (STP) v1.0.
- docs/api-contract.md in the repository: the agreed JSON shapes and in-process function signatures.
- The four member plans (v2) that assign stories, owners and dates.

3. Architecture

3.1 Goals and constraints

Goals: a modular, maintainable codebase that four people can build in parallel; secure handling of accounts and tokens; correct, atomic bookings; responsive search (< 1 s), booking (< 1 s) and recommendations (< 2 s); at least 70% unit-test coverage per module; automatic build, test and deployment; a standalone product with its own database and no outside services.

Constraints: two sprints (30 Oct – 3 Nov 2026 and 3 Nov – 8 Nov 2026); a team of four; Python stack; free hosting tier; one GitHub repository with a protected main branch.

3.2 Stakeholders and concerns

Stakeholder	Main concerns
Members (readers)	Easy search, reliable bookings, relevant recommendations, privacy of their data
Administrators (library staff)	Accurate catalog, control over copies, no double booking
Development team	Clear module boundaries, stable interfaces, parallel work, fast feedback from CI
QA Lead	Testability, traceability from requirements to tests, repeatable test data
Instructor and TAs	Requirements coverage, proper Jira tracking, a working CI/CD pipeline, a working demo

3.3 Component (UML) diagram

Figure 1 shows the components and their interfaces; Figures 2 and 3 show the class diagrams of the domain model and of the recommendation design. The diagrams are on landscape pages so they stay readable.

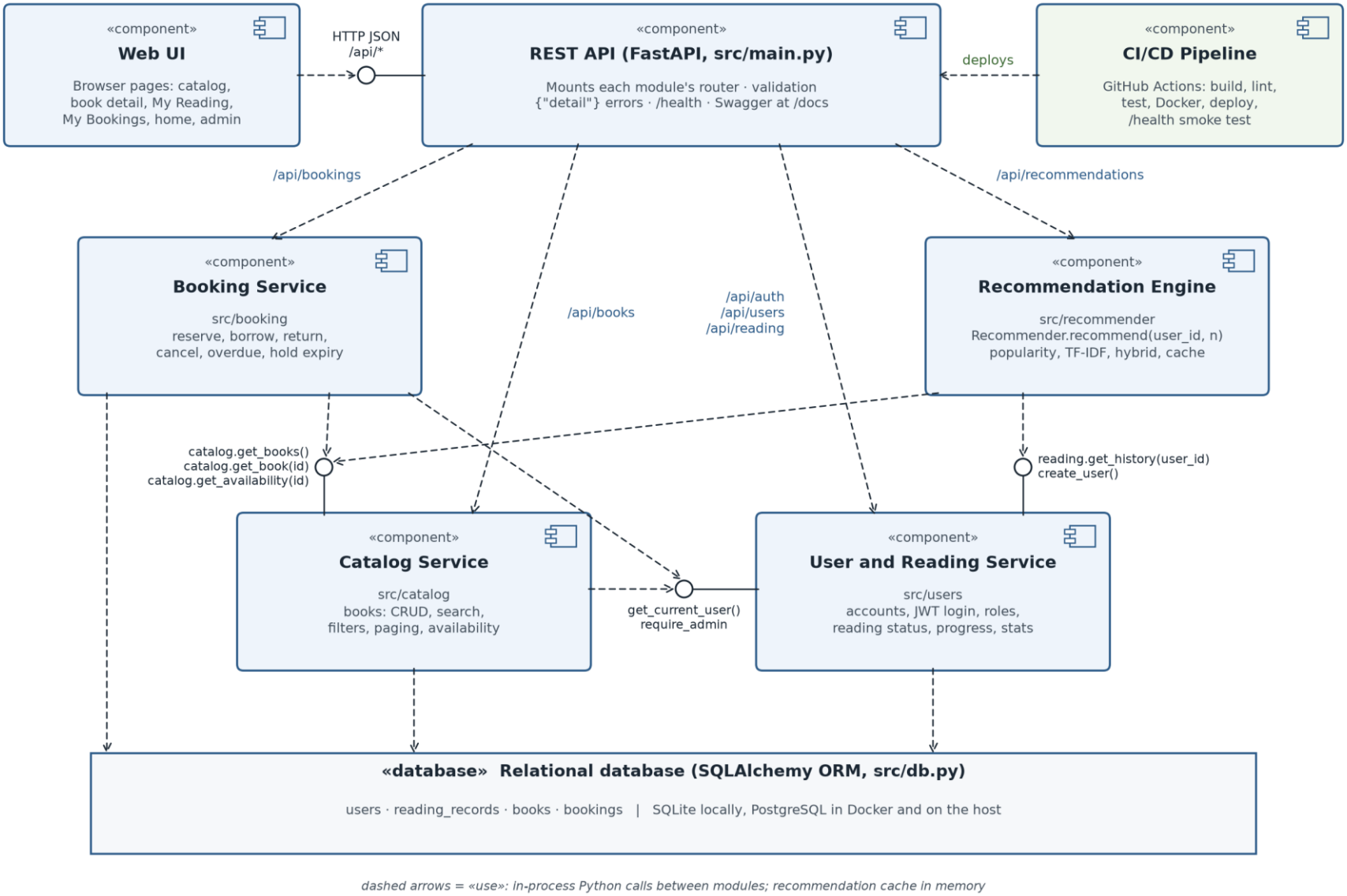


Figure 1. Component diagram: provided interfaces (lollipops) and «use» dependencies between components.

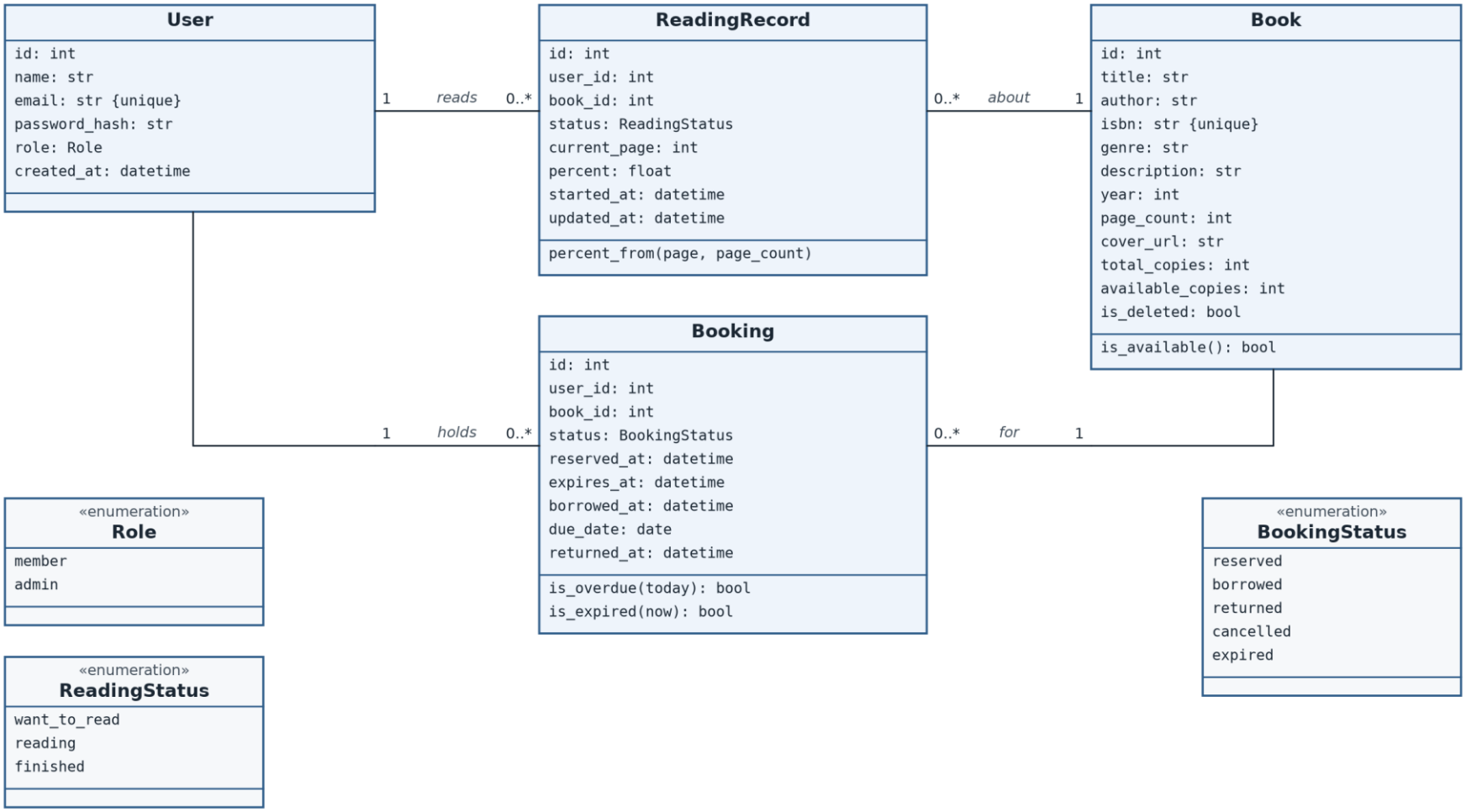
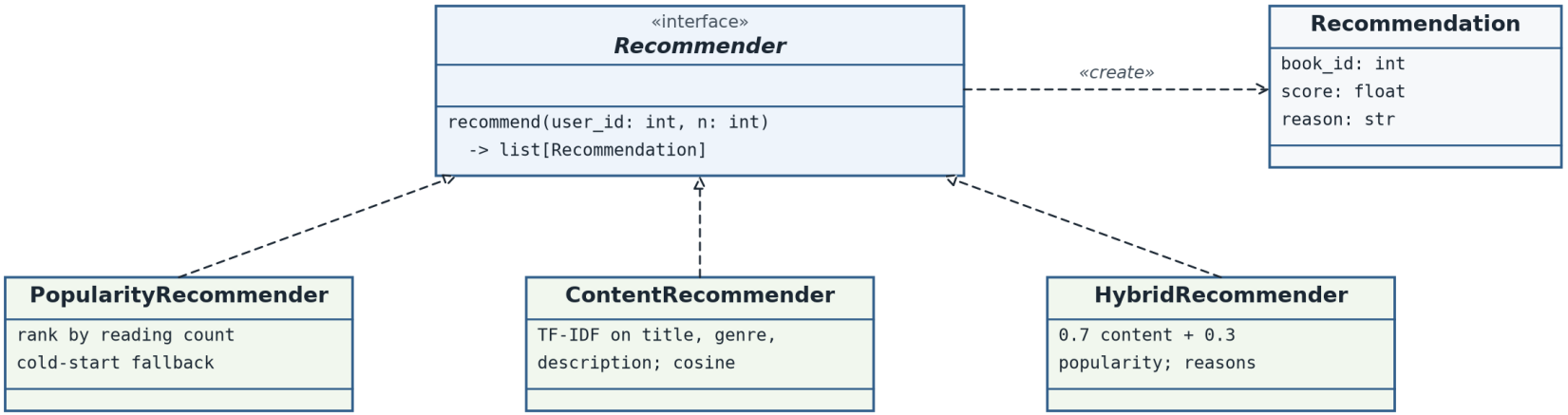


Figure 2. Class diagram of the domain model (the four database tables and their enumerations).



Dashed arrows with hollow heads: realisation of the Recommender interface (NFR-R2: algorithms are swappable).

Figure 3. Class diagram of the recommendation design: one interface and three swappable algorithms.

3.4 Component descriptions

Component	Responsibility	Provides	Requires	Owner
Web UI	Pages for catalog, book detail, My Reading, My Bookings, home, profile and admin; keeps the login token	Screens	HTTP JSON API	M2 (app shell) with each module owner
REST API (src/main.py)	Builds the FastAPI app, mounts every module's router, common error format, /health, API docs	HTTP JSON API	Module routers	M3
Catalog Service (src/catalog)	Book CRUD with validation, search, filters, paging, soft delete, availability	/api/books; catalog.get_books(), get_book(id), get_availability(id)	get_current_user, require_admin; database	M2
User and Reading Service (src/users)	Registration, login (JWT), roles, first admin, profile, reading status, progress, history, statistics	/api/auth, /api/users, /api/reading; get_current_user, require_admin, create_user, reading.get_history(user_id)	Database; book page counts from Catalog	M3
Booking Service (src/booking)	Reserve, borrow, return, cancel; limit of 5; overdue and 2-day hold expiry; the only code that changes available copies	/api/bookings	get_current_user; catalog.get_availability(id); database	M4
Recommendation Engine (src/recommender)	Popularity, TF-IDF content and hybrid scoring, reasons, cold start, cache, admin refresh	/api/recommendations; Recommender.recommend(user_id, n)	catalog.get_books(); reading.get_history(user_id)	M1
Database (src/db.py)	Engine, sessions and the Base class; tables users, reading_records, books, bookings	SQL through the ORM	–	M3 (setup); each table's owner
CI/CD pipeline	Build, lint, test with coverage gate, Docker image, deploy on merge to main, /health smoke test	Deployed app	GitHub Secrets; hosting platform	M3

3.5 Chosen architecture pattern and rationale

DLBS uses a **layered modular monolith**. Inside one application there are three layers (web UI, REST API, modules over one database), and the business logic is split into four modules that talk only through small published interfaces. This suits a team of four on a two-sprint timeline: each person owns a module, interfaces can be stubbed on day one, and there is only one thing to build, test and deploy. It also makes DLBS a standalone product: one deployable unit with its own database and no outside services.

ADR	Decision	Alternatives considered	Reason
ADR-1	Modular monolith	Microservices; one unstructured monolith	Microservices add network calls, separate deployments and distributed transactions that two sprints cannot absorb; an unstructured monolith would block parallel work.
ADR-2	Modules call each other in-process (Python functions)	HTTP calls between modules	Faster (helps NFR-R1), no tokens or retries between modules, easy to stub in tests.
ADR-3	Stateless JWT login	Server-side sessions	Works the same locally, in CI and on the host; no session store to run.
ADR-4	SQLAlchemy ORM over SQLite and PostgreSQL	Raw SQL; one database everywhere	Same code in tests (SQLite in memory) and production (PostgreSQL); parameterised queries by default.
ADR-5	Recommendation algorithms behind one Recommender interface	One fixed algorithm	The baseline ships in Sprint 1 and the hybrid replaces it in Sprint 2 without touching callers (NFR-R2).

3.6 Technology stack and data stores

Layer	Technology	Why
Language	Python 3.11	One language for the API and the machine-learning code

Layer	Technology	Why
Web framework	FastAPI with Uvicorn	Request validation, automatic OpenAPI docs at /docs, dependency injection for get_current_user
Data access	SQLAlchemy 2	Same models on SQLite and PostgreSQL; safe parameterised queries
Databases	SQLite (development, tests); PostgreSQL (Docker, deployed host)	Zero setup locally; robust transactions in production
Security	bcrypt; PyJWT (HS256, 60-minute tokens)	Standard password hashing and signed tokens
Machine learning	scikit-learn (TfidfVectorizer, cosine similarity), pandas	Proven, simple content-based recommendation
User interface	HTML, CSS and JavaScript pages served by the app	No separate front-end build; pages call the REST API
Testing and quality	pytest, pytest-cov, FastAPI TestClient, ruff	Unit and API tests with coverage; PEP 8 linting
CI/CD and hosting	GitHub Actions, Docker, docker-compose, free cloud host (Render or Railway)	Build, test and deploy on every merge; health-checked deployment
Planning	Jira (sprints, burndown), GitHub Project mirror	Course requirement

Table 1. Data stores.

Table	Key columns	Owner	Notes
users	id, name, email (unique), password_hash, role, created_at	Users	role is member or admin (check constraint)
reading_records	id, user_id, book_id, status, current_page, percent, started_at, updated_at	Users	one record per user and book
books	id, title, author, isbn (unique), genre, description, year, page_count, cover_url, total_copies, available_copies, is_deleted, created_at	Catalog	indexes on title, author, isbn; only Booking changes available_copies
bookings	id, user_id, book_id, status, reserved_at, expires_at, borrowed_at, due_date, returned_at	Booking	status reserved, borrowed, returned, cancelled or expired
(memory)	TF-IDF matrix, popularity scores, per-user result cache	Recommendation	rebuilt on admin refresh; not persisted

3.7 Risks and mitigations

Risk	Mitigation
Late integration between modules	API contract and stubs from 29 Oct; stubs swapped for real functions early in Sprint 2; first cross-module smoke test on 3 Nov
Two people changing the shared database	One shared src/db.py; each table has one owner; every change goes through a reviewed pull request
Goodbooks-10k lacks genre, description and page count	Genre from top tags, description from title, author and tags, page count generated with a fixed seed; a 20-book sample for tests
Two members reserving the last copy at once	Conditional UPDATE inside one transaction (SD-2); concurrency test TC-B11
Slow recommendations on 10,000 books	TF-IDF built once on refresh; per-user cache; candidate filtering before scoring
Free host down or slow during the demo	docker-compose run locally as a fallback; recorded demo video; /health smoke test after every deploy
Compressed timeline (Sprint 2 ends 8 Nov)	High-priority requirements first; stretch collaborative filtering only if on schedule
Secrets leaking into the repository	.env git-ignored; GitHub Secrets for deploy tokens; review checklist

3.8 Traceability to requirements

Requirements	Component	Design element
FR-C1–C9, NFR-C1–C3	Catalog Service; Web UI (C6, C9)	books table, /api/books, indexes, catalog functions

Requirements	Component	Design element
FR-U1–U8, NFR-U1–U4	User and Reading Service	users and reading_records, JWT, get_current_user, SD-1
FR-B1–B10, NFR-B1–B3	Booking Service	bookings table, state rules (4.1), conditional update, SD-2
FR-R1–R6, NFR-R1–R3	Recommendation Engine	Recommender interface, HybridRecommender, cache, SD-3
SR-01–SR-07	User and Reading Service, every router, CI/CD	Section 3.9
NFR-G1	CI/CD pipeline	Sections 3.6 and 4.4
NFR-G2	Web UI	Section 4.5

3.9 Security architecture

Threat model (STRIDE) for the main assets: accounts and passwords, reading histories, bookings and copy counts, the catalog, and deployment secrets.

Threat	Example	Mitigation	Requirement
Spoofing	Logging in as someone else	bcrypt hashes; signed 60-minute JWT; the same error for wrong email or password	SR-01, SR-02
Tampering	Editing a token's role; changing another member's booking; SQL injection	Signed tokens; role and ownership checks; server-side validation; ORM parameterised queries	SR-02 to SR-05
Repudiation	A member denies borrowing a book	Bookings keep user_id and timestamps for every state; state changes are logged	FR-B7
Information disclosure	Leaking passwords, other members' data or secrets	Hashes only; ownership checks; HTTPS; secrets in environment variables and GitHub Secrets; no secrets in logs	SR-04, SR-06, SR-07
Denial of service	Huge page sizes or request floods	Page size and limit capped at 50; indexes; recommendation cache; host rate limits	NFR-C1, NFR-R1
Elevation of privilege	A member calling admin endpoints	require_admin on every admin route (403); role check constraint; the first admin only from environment variables	SR-03, FR-U3

4. Design

4.1 Design overview

Every module follows the same three-layer layout, so the code looks the same everywhere:

- **routes.py**: the FastAPI router; validates the request, gets the user with get_current_user, calls the service and shapes the response.
- **service.py**: the business rules; the only place that changes data; exposes the in-process functions other modules call.
- **models.py**: SQLAlchemy tables built on the shared Base in src/db.py.

Booking state rules. Allowed transitions: reserved → borrowed (borrow), borrowed → returned (return), reserved → cancelled (cancel), reserved → expired (2 days without borrowing). Any other transition is rejected with 409. Overdue and expired are worked out when a booking is read, so no scheduled job is needed.

Recommendation algorithm. Popularity counts how many members have read each book. For content similarity, each book's title, genre and description become a TF-IDF vector; a member's profile is the average of the vectors of the books they read, weighted by progress. Unread, not-removed books are ranked by 0.7 × cosine similarity + 0.3 × popularity, and the reason names the most similar book the member has read. A member with no history gets the most popular books (cold start). Quality is measured with Precision@5 and Recall@5 on held-out history.

4.2 UML sequence diagrams

Three flows that cross module boundaries are shown on the following landscape pages: logging in and calling a protected endpoint (SD-1), reserving a book (SD-2) and showing recommendations (SD-3).

SD-1 Log in and open a protected page (FR-U2, FR-U7, NFR-U1, NFR-U2)

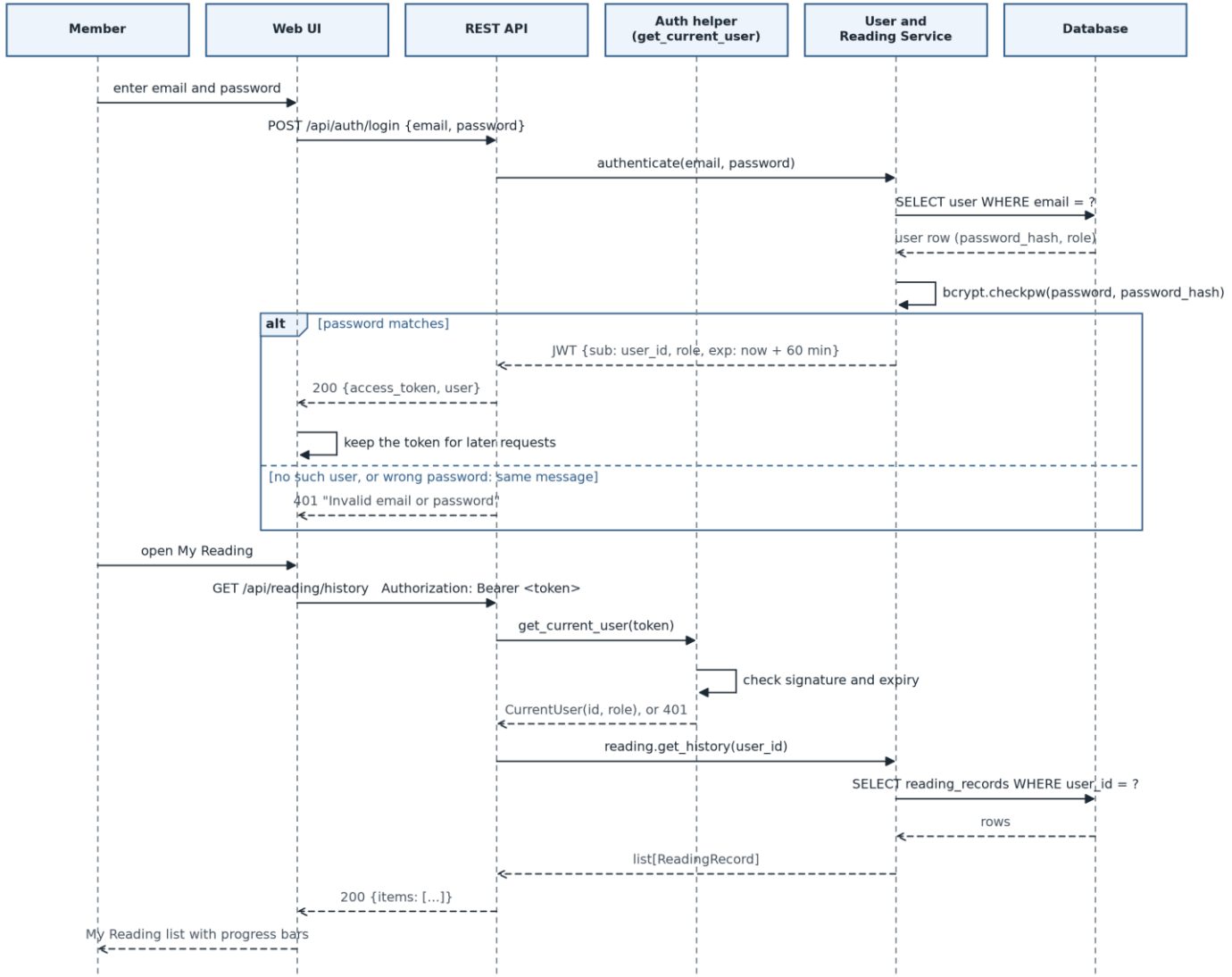


Figure 4. SD-1: log in and open a protected page.

SD-2 Reserve a book (FR-B1, FR-B5, FR-B8, FR-B9, NFR-B1)

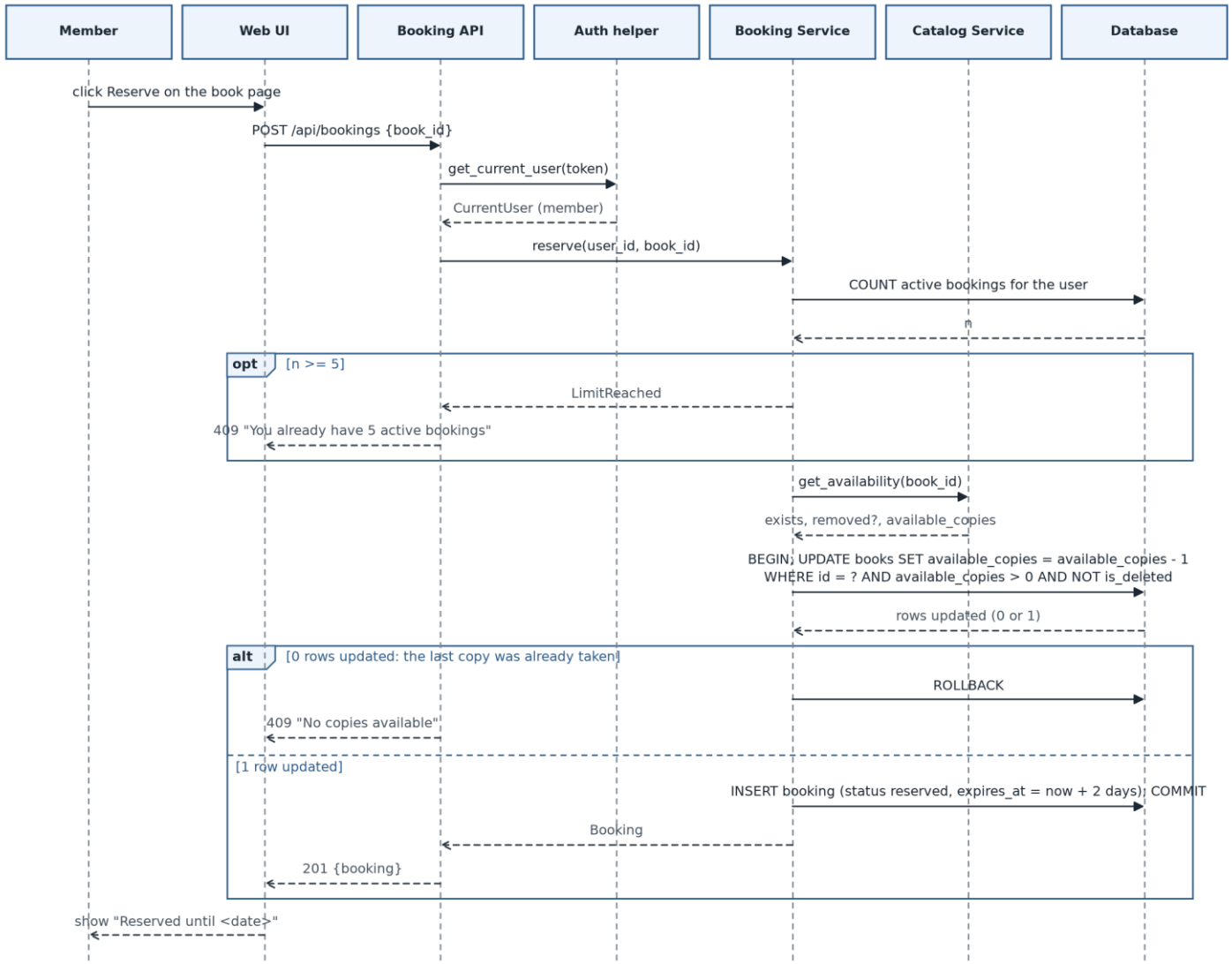


Figure 5. SD-2: reserve a book in one atomic transaction.

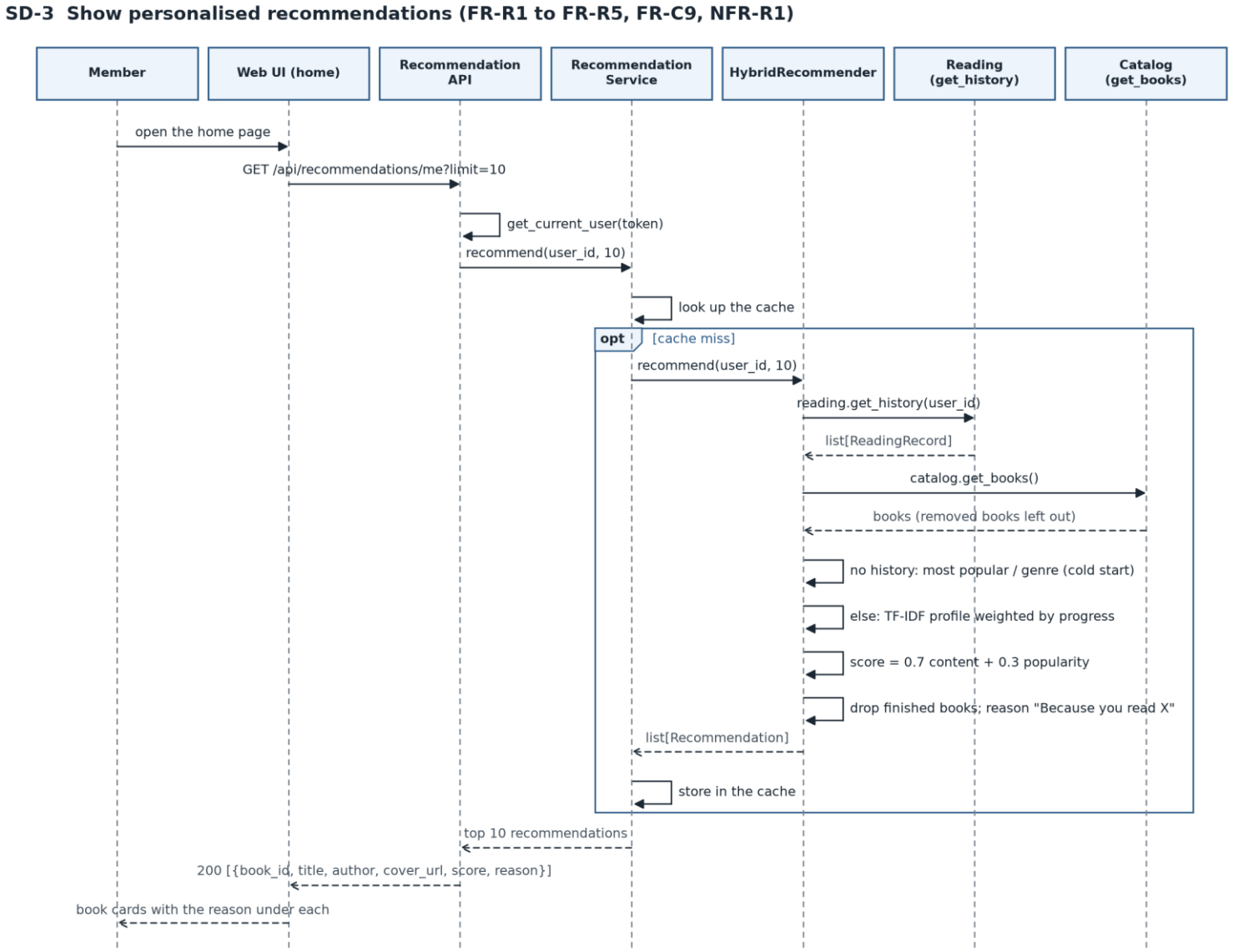


Figure 6. SD-3: show personalised recommendations.

4.3 API design

All endpoints take and return JSON. Protected endpoints need Authorization: Bearer <token>. Errors return {"detail": "message"}. The full shapes are in docs/api-contract.md.

User and Reading Service

Endpoint	Request	Response	Errors
POST /api/auth/register	{name, email, password}	201 {id, name, email, role, created_at}	409 email exists; 422 invalid
POST /api/auth/login	{email, password}	200 {access_token, token_type, expires_in, user}	401 invalid email or password
POST /api/auth/logout	–	204	401
GET /api/users/me	–	200 {id, name, email, role}	401
PUT /api/users/me	{name}	200 profile	401; 422
PUT /api/reading/status	{book_id, status}	200 reading record	401; 404; 422
POST /api/reading/progress	{book_id, current_page} or {book_id, percent}	200 reading record	401; 404; 422 out of range
GET /api/reading/history	?status=	200 {items: [reading record]}	401
GET /api/reading/stats	–	200 {books_finished, books_reading, books_want_to_read, pages_read}	401

Booking Service

Endpoint	Request	Response	Errors
POST /api/bookings	{book_id}	201 booking (reserved, expires_at)	401; 404; 409 no copies, removed or limit of 5
POST /api/bookings/{id}/borrow	–	200 booking (borrowed, due_date)	401; 404; 409 wrong state
POST /api/bookings/{id}/return	–	200 booking (returned)	401; 404; 409 wrong state
DELETE /api/bookings/{id}	–	200 booking (cancelled)	401; 404; 409 wrong state
GET /api/bookings/me	–	200 {items: [booking with overdue flag]}	401

Catalog Service

Endpoint	Request	Response	Errors
GET /api/books	?q=&genre=&available=&page=&size=	200 {items: [book], page, total}	422 bad parameters
GET /api/books/{id}	–	200 book	404
POST /api/books (admin)	book fields	201 book	401; 403; 409 duplicate ISBN; 422
PUT /api/books/{id} (admin)	changed fields	200 book	401; 403; 404; 409 copies below loans
DELETE /api/books/{id} (admin)	–	204 (soft delete)	401; 403; 404
GET /api/books/{id}/availability	–	200 {book_id, available_copies}	404

Recommendation Engine

Endpoint	Request	Response	Errors
GET /api/recommendations/me	?limit=10	200 [{book_id, title, author, cover_url, score, reason}]	401

Endpoint	Request	Response	Errors
POST /api/recommendations/refresh (admin)	–	202 {status}	401; 403
GET /health	–	200 {status: "ok"}	–

In-process interfaces

Signature	Returns	Used by
catalog.get_books()	All books that are not removed	Recommendation Engine
catalog.get_book(id) / catalog.get_availability(book_id)	One book / exists, removed and available_copies	Booking Service
reading.get_history(user_id)	list[ReadingRecord], newest first; [] for unknown users	Recommendation Engine
get_current_user() / require_admin	CurrentUser(id, name, email, role); 401 or 403	Every router
create_user(name, email, password, role)	The new User	Registration, seed script
Recommender.recommend(user_id, n)	list[Recommendation(book_id, score, reason)]	Recommendation API

Book JSON: {id, title, author, isbn, genre, description, year, page_count, cover_url, total_copies, available_copies}. Booking JSON: {id, book_id, status, reserved_at, expires_at, borrowed_at, due_date, returned_at, overdue}. Reading record JSON: {book_id, title, author, genre, status, current_page, percent, started_at, updated_at}.

4.4 Error handling, logging and monitoring

- One error format, {"detail": "message"}, with 401 (not logged in), 403 (role), 404 (missing), 409 (conflict or wrong state) and 422 (validation). Messages are clear and never reveal whether an email exists.
- Python logging at INFO for state changes (account created, booking reserved or returned, model refreshed) and WARNING for rejected logins and permission failures. Passwords, tokens and secrets are never logged.
- GET /health returns 200 when the app and database respond. The deploy job calls it after every deployment and fails the pipeline if it does not answer; the host also uses it as a health check.
- GitHub Actions keeps the history of builds, tests and coverage reports for every pull request.

4.5 UX design

- An app shell with a navigation bar: Home, Catalog, My Reading, My Bookings, Profile, Admin (admins only), and Log in or Log out.
- A reusable book card (cover, title, author, availability badge, or the recommendation reason) used on the home page and in the catalog.
- Progress bars for reading, text status badges for bookings (never colour alone), and due dates shown in full.
- Labelled form fields with inline error messages taken from the API's detail text.
- Responsive layout for phone and desktop widths, visible keyboard focus, and alternative text on covers (NFR-G2).

4.6 Open issues and next steps

- Item-based collaborative filtering as a third Recommender, only if the evaluation is done early.
- Server-side token revocation on logout (today the client discards the token).
- Due-date reminders by e-mail, fines and password reset are out of scope for this release.
- Database migrations with Alembic if the schema must change after the first deployment.
- Each module owner confirms the final JSON shapes in docs/api-contract.md.

5. Appendices

5.1 Glossary

DLBS, ADR, JWT, ORM, STRIDE, TF-IDF, CI/CD, RTM: see section 1.4 of this document and of the SRS.

5.2 References

- ISO/IEC/IEEE 42010:2022, Architecture description.
- OWASP Top 10 (2021) and OWASP Application Security Verification Standard 4.0.
- FastAPI, SQLAlchemy 2.0 and scikit-learn documentation.
- Goodbooks-10k dataset (Zając, 2017).
- Course templates for the SRS, SAD and Software Test Plan.

5.3 Tools

draw.io and Python (matplotlib) for UML diagrams; Swagger UI at /docs for the API; GitHub and GitHub Actions; Jira; Visual Studio Code.